4.3 Memory Blocks

We define memory $M = \text{Time} \times (\text{BlockID} \nrightarrow \text{Block}) \times (\text{CallID} \nrightarrow \text{Time} \uplus \{\textbf{None}\})$ as a triple of a time stamp, a map from logical block ids to memory blocks, and a map recording the time stamp of each function call (indexed on *cid* of physical pointers). When a new memory block is created (e.g., with **malloc** or **alloca**) or deallocated, the time stamp is incremented by one.

A memory block is a tuple (t, r, n, a, c, P), where t is the block type (e.g., stack or heap allocated), r is the life range of the block, n is the block size in bytes, a is the alignment, c the contents of the block (the actual data), and P has the addresses of the block.

When a block is deallocated (e.g., with **free** or on function exit), it is not deleted from memory. [3] Instead, we set the end of the lifetime range to the current memory time stamp, and increment it as well. FREE-LOGICAL in Fig. 5 shows the semantics of **free**. If a physical pointer $\mathsf{Phy}(o, s, I, cid)$ is given to **free**, it is equivalent to freeing a dereferenceable block whose base address is o. Double-freeing a block or **free** with a logical pointer with non-zero offset is UB. **free** with NULL is no-op.

Memory allocation functions reserve at least two blocks: The block actually observed by the program, and N additional *twin blocks*. The number N of twin blocks is a parameter of the semantics. (We will discuss in Section 4.12 why $N = 1$ might not be enough.) The block's base addresses in address space s are stored in $P(s)$, a sequence of physical addresses: $P(s)_0$ is the actual base address used and observed by the program, while the remaining addresses in the sequence are the base addresses of the twin blocks.

Crucially, we maintain the invariant that for P, P' of any pair of different live (non-deallocated) blocks, the address ranges $[P(s)_i, P(s)_i + n)$ and $[P'(s)_j, P'(s)_j + n)$ are disjoint. Thus, **malloc** reserves space for both the block and its twins (Fig. 5). The rest of the semantics only depends on $P(s)_0$ and ignores the remaining base addresses.

In Fig. 5, notation $[\![op]\!]_R$ represents the evaluation of an instruction's operand:

$$[\![v]\!]_R = \begin{cases} R(v) & \text{if } v \text{ is a register} \\ v & \text{if } v \text{ is a constant or } v = \textbf{poison} \end{cases}$$

4.4 Pointer Arithmetic

LLVM IR has a single instruction for pointer arithmetic, **getelementptr**, or **gep** for short. In Fig. 5 we show the semantics of several cases. For a logical pointer, the result is also a logical pointer where only the offset is updated (GEP-LOGICAL). Likewise for physical pointers (GEP-PHYSICAL).

Function $\text{inbounds}_M(l, o)$ checks if a given offset o is within bounds of object l in memory $M = (\tau, m, C)$. If $m(l) = (t, r, n, a, c, P)$, $\text{inbounds}_M(l, o)$ is true iff $0 \leq o \leq n$.

When **gep** has the **inbounds** tag (e.g., when compiling C/C++ code or most cases in safe languages), the compiler can assume that the input pointer is valid (within bounds of some object or else one element past the end) and also that the resulting pointer is valid. For logical pointers, the bounds checking is immediate (GEP-INBOUNDS-LOGICAL). If either of the inbounds conditions fails, the result is **poison** (not shown).

If the pointer is physical, we use deferred bounds checking: input and output offsets are added to I. They are checked to be inbounds only when the pointer is dereferenced. As seen in Section 3.1, this allows free movement of **gep** instructions since they do not depend on the memory state.

[3] Note that memory is deallocated at run time as expected. Also, in our semantics addresses can be reused because allocation functions allocate blocks with addresses that are disjoint from all other *live* blocks only.